

HSCTF 2020 Crypto - Chonky E

deuterium — 2020-06-06

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/HSCTF/crypto/Chonky'E/2020-06-06-HSCTF-2020-Chonky-E.html>

Abstract

CTF writeup: HSCTF 2020's Chonky E skimps on the private exponent: Boneh-Durfee cracks RSA, and shared primes take Schmidt-Samoa down with it.

Chonky E

Challenge

154 Solves

×

Chonky E

100

rsa crypto

E

Note: $P > Q$

Author: AC

 ChonkyE.txt

Flag

Submit

Allen and Jason rely on two different cryptosystems to keep their information secure.
Allen uses the RSA cryptosystem, given by the following public key:

```
e =
91043118409828550796773745518585981151180206101005135117565865602978722878478494447048
78355757181398052564372532337748824983886089778468392702990618894700114963210151336725
82673299616840346612528664849819260550873861900154329646089279476464761932518203547386
40453947833718397360834701566765504916472450194494897616371452996381159817427887623703
63913329035852049841904917594158467880270160699509924124592688417298500483980127000558
30305142865619718250477194214870045697526384689076091102857390832796297473109530865358
89932550905065172805818862336335628248528993024112446002398466115161473573451161053837
400091893285717
```

```
n =
15674904755858301396051326735176947991511044041144807841259056579703153362250981335209
31196368355119772530338543884668541427537761460925878254404451820082373252620126980344
19137157047927918635897378973846177552961727126115560551970797370239385129543828686170
77432330693320248172888401942042236036084959298381840515447336979018163647213774186544
02333839565710811229822236026678536687543383600082790023255764955738475683015843655144
17593244726435632222027817410359417329310347952169273512510934251453361933794586716533
95048997343639383418950545095662228621681944077716280479843233093335705817588567418458
2816364542591313
```

Jason uses the Schmidt-Samoa cryptosystem. Although a public key has not been recovered, we know that Allen and Jason share the same primes (p, q) .

A ciphertext was found on Jason's computer that reads:

```
16267540901004879123859424672087486188548628828063789528428674467464407443871599865993
33755586953048624113913865064183837741973489780138088362989416635322528800614821045367
70237506881751923172414404577687882672704228570605342616745387557432448311524709951249
62736526978165448560149498403762447372653982922113772190234143253450918953235222315161
96453931103265962867041749617412348304543935984636004877416433725782939834568663509186
23062044556873474439589314412255008564083317952613290350725856054044164739872800379591
84981453888701567175803979981461050532113072292714696752692872526424122826696681194705
56339116113742670369090073370686684236305596785644376521572339855552212690974923675933
29648732219739703688775654106248951604386950064320215290718668819051344944892668010049
03504121740435965696128048690741210812963902631391765192187570107372453917327060678806
282122942318369245760773848604249664378721970318257356486696764545
```

What are the contents of this message?

Since both crypto use same p and q doesn't reveal a method to recover p or q or the plaintext somehow, second BIG hint is the name of the challenge :- chonkyE

Big E could lead to two attacks, and Boneh Durfee attack is more powerful of the two.

Here is the evergreen [sage script](#) for the same. I modified it a bit (by fixing annoying python2 print statements) to run on sage 9 (python3).

```
d =
48018206241103005673812641526303609844001011980066627783381059991900254490396537225463
63948393959163699344836724430590700225590643966670154013435626235133
```

We can recover p , q from n , e , d . The most convenient way according to me is to use RSA module from Crypto.PublicKey

```
from Crypto.PublicKey import RSA
import gmpy2

n =
```

```

15674904755858301396051326735176947991511044041144807841259056579703153362250981335209
31196368355119772530338543884668541427537761460925878254404451820082373252620126980344
19137157047927918635897378973846177552961727126115560551970797370239385129543828686170
77432330693320248172888401942042236036084959298381840515447336979018163647213774186544
02333839565710811229822236026678536687543383600082790023255764955738475683015843655144
17593244726435632222027817410359417329310347952169273512510934251453361933794586716533
95048997343639383418950545095662228621681944077716280479843233093335705817588567418458
2816364542591313
e =
91043118409828550796773745518585981151180206101005135117565865602978722878478494447048
78355757181398052564372532337748824983886089778468392702990618894700114963210151336725
82673299616840346612528664849819260550873861900154329646089279476464761932518203547386
40453947833718397360834701566765504916472450194494897616371452996381159817427887623703
63913329035852049841904917594158467880270160699509924124592688417298500483980127000558
30305142865619718250477194214870045697526384689076091102857390832796297473109530865358
89932550905065172805818862336335628248528993024112446002398466115161473573451161053837
400091893285717
d =
48018206241103005673812641526303609844001011980066627783381059991900254490396537225463
63948393959163699344836724430590700225590643966670154013435626235133

rsa = RSA.construct((n, e, d))
p = rsa.p
q = rsa.q

# rest is just implementation of Schmidt-Samoa

ct =
16267540901004879123859424672087486188548628828063789528428674467464407443871599865993
33755586953048624113913865064183837741973489780138088362989416635322528800614821045367
70237506881751923172414404577687882672704228570605342616745387557432448311524709951249
62736526978165448560149498403762447372653982922113772190234143253450918953235222315161
96453931103265962867041749617412348304543935984636004877416433725782939834568663509186
23062044556873474439589314412255008564083317952613290350725856054044164739872800379591
84981453888701567175803979981461050532113072292714696752692872526424122826696681194705
56339116113742670369090073370686684236305596785644376521572339855552212690974923675933
29648732219739703688775654106248951604386950064320215290718668819051344944892668010049
03504121740435965696128048690741210812963902631391765192187570107372453917327060678806
282122942318369245760773848604249664378721970318257356486696764545

# N = p**2*q
# d = N^-1 mod lcm(p-1, q-1)
# m = c^d mod pq

N = p*q*q # since this one works
lcm_val = gmpy2.lcm(p-1, q-1)
D = gmpy2.invert(N, lcm_val)
m = pow(ct, D, p*q)
print(bytes.fromhex(hex(m)[2:]))
# flag{remarkably_superb_acronym}

```

Interactive controls are omitted from this PDF. See the canonical HTML version.